Architecture Modeling Track

Lab T-AM1: Design-Space Exploration

Trace-Driven Simulation and Model–RTL Correlation

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Drive the T-MOD cache model with address traces to explore hundreds of cache configurations in minutes, prove that the model's predictions match the R4 RTL exactly on the same traces, and turn the results into a cache configuration recommendation under an area budget.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Lab T-MOD; from the common core Labs R4 (`dcache` and its performance counters) and P3 (area of the cache configurations)
Designs	Trace generators and traces (`dv/model/traces/`, new); `tb_explore`, `tb_replay` and scripts (`dv/model/explore/`, new); `cache_model_pkg` and `dcache` used unchanged
Reused in	T-AM2 (traces, replay testbench and correlation flow are extended with timing)

Contents

1 Overview **3**

2 Traces **4**

2.1 Why Traces . . . 4

2.2 Trace Format . . . 4

2.3 Generators . . . 5

3 Model-Only Exploration **6**

3.1 The Exploration Testbench . . . 6

3.2 Cold Start and Warm-Up . . . 7

3.3 The Three-C Classification . . . 7

3.4 Plots . . . 8

3.5 Questions . . . 8

4 Model–RTL Correlation **9**

4.1 What Must Match, and Why Exactly . . . 9

4.2 The Trace-Driven RTL Testbench 9
4.3 When They Disagree 10
4.4 Model Speed versus RTL Speed 11

5 Performance and AMAT **11**

6 The Recommendation Memo **12**
6.1 Area Data 12
6.2 The Memo 12

7 Requirements and Tasks **12**

8 Repository and Flow **13**

9 Deliverables **14**

10 Assessment **15**

11 Review Questions **15**

12 References **15**

1 Overview

Every cache parameter is a trade-off. Longer lines exploit spatial locality but waste bandwidth on random accesses; more lines cost area; associativity removes conflict misses but lengthens the critical path through the tag comparison (Lab P3). Architects decide these parameters *before* the RTL is written, and they decide them with models, because a model answers a "what if" question in seconds that would take an RTL change, a regression and a synthesis run to answer otherwise.

A model is only useful if it is right. Modeling teams therefore spend as much effort on **correlation** — showing that the model and the RTL agree on the same workload — as on the model itself. A model that disagrees with the RTL by an unexplained 10% cannot be used to choose between two configurations that differ by 5%.

Test term — Model correlation test. The same trace file is replayed on the transaction-level model and on the RTL, and their results are compared: here the hit, miss and write-back counts, which must match exactly (T-AM2 adds cycles, which must match within a stated error). A pass proves that the model can be trusted for the configurations and workloads that were correlated — and only for those. It is the gate between "we have a model" and "we use the model for decisions".

In this lab you build the standard trace-driven exploration flow (Figure 1): generators write address traces to files; a model-only testbench replays every trace on many configurations and writes a CSV file; Python scripts turn the CSV into plots. The same trace files are then replayed on the R4 RTL, and the hit, miss and write-back counts must match the model's prediction *exactly* — for a functional quantity such as a miss count there is no acceptable error. Finally you combine the hit-rate results with the area numbers from P3 and write a one-page recommendation, as a performance engineer would for an architecture review.

Where this lab sits. It uses the verified model from Lab T-MOD without changing it, the R4 RTL and its performance counters without changing them, and the area results of Lab P3. It predicts *counts*, not cycles: time is added in Lab T-AM2, which reuses the traces, the replay testbench and the correlation scripts of this lab.

By the end of this lab you should be able to:

1. Define a trace format and write reproducible trace generators for synthetic and mixed access patterns.
2. Run a model-only parameter sweep and present the results as CSV data and plots.
3. Explain hit-rate results with locality, the three-C classification of misses and the behavior of replacement policies.
4. Replay one trace on the model and on the RTL, and correlate their event counts exactly; find the first divergent access when they differ.
5. Quantify the speed advantage of a transaction-level model over RTL simulation and explain what it means for architecture work.
6. Combine performance and area data into a justified configuration recommendation under a budget.

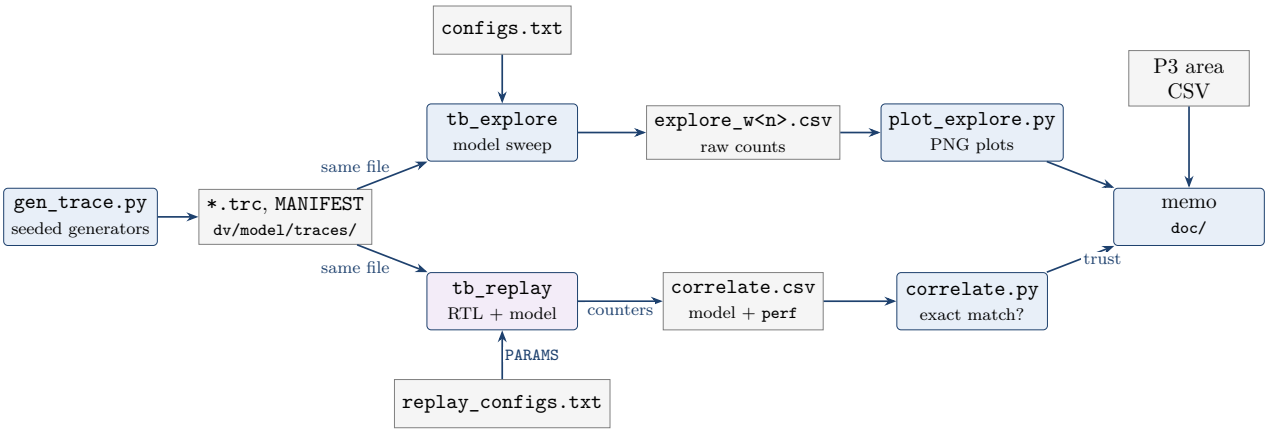


Figure 1: The exploration and correlation flow of this lab. Upper path: trace generators → traces → model sweep → CSV → plots. Lower path: the *same* trace files (checked by their SHA-256) → RTL testbench → performance counters → exact correlation. Only a correlated model feeds the memo.

2 Traces

2.1 Why Traces

A trace decouples *what the program does* from *what the cache does*. The same file can be replayed on any model configuration and on the RTL, so differences in the results can only come from the cache, not from the stimulus. Real performance teams capture traces from instruction-set simulators (Spike, QEMU) or from RTL runs of real software; in this lab you generate synthetic traces whose behavior you can predict, plus one mixed trace that looks more like a program.

2.2 Trace Format

A trace is a text file, one access per line, word addresses and data in hexadecimal:

```
# trace=loop  generator=gen_trace.py v1  seed=1  addr_w=10  n=20000
# params: base=0x000 size=80 wr_pct=0
R 000
R 001
W 0a3 deadbeef
R 002
```

- `R addr` is a read; `W addr data` is a write. Data widths follow `DW`; addresses must fit in `ADDR_W` bits (the reader checks and fails with the line number).
- Lines starting with `#` are comments. The first comment line is mandatory and records the generator, its version, the seed and every parameter: a trace must be reproducible from its header alone.
- There is no timing information: accesses are issued back to back. The end of a trace implies a flush, so that write-backs of dirty lines are counted in both model and RTL.

The reader is a small class shared by every testbench of the track:

```
class trace_reader #(int ADDR_W = 10, int DW = 32);
  typedef cache_model_pkg::cpu_txn #(ADDR_W, DW) txn_t;

  // returns the trace name of the header; lineno[k] = file line of access k
```

```
  static function string load(string fname, ref txn_t q [$], ref int unsigned lineno [$
    ]);
    int fd = $fopen(fname, "r");
    string line; int unsigned ln = 0;
    if (fd == 0) $fatal(1, "cannot open trace %s", fname);
    while ($fgets(line, fd)) begin
      byte op; logic [63:0] a, d; int n;
      ln++;
      if (line.len() < 2 || line[0] == "#") continue;   // (line 1: header check,
    omitted)
      n = $sscanf(line, "%c %h %h", op, a, d);
      if (!(op inside {"R", "W"}) || n < 2 || (op == "W" && n != 3))
        $fatal(1, "%s:%0d: bad trace line '%s'", fname, ln, line);
      if ((a >> ADDR_W) != 0)          // read wide, then check: %h into ADDR_W bits
    truncates
        $fatal(1, "%s:%0d: address %h does not fit ADDR_W=%0d", fname, ln, a, ADDR_W);
      q.push_back(txn_t::make(op == "W", a[ADDR_W-1:0], d[DW-1:0]));
      lineno.push_back(ln);
    end
    $fclose(fd);
    // ... compare the number of accesses with the header's n=, return its trace= field
  endfunction
endclass
```

Two SystemVerilog details bite here. `$sscanf` with `%h` into an `ADDR_W`-bit variable silently drops the upper bits, so the address is read into 64 bits and checked. And `"\r"` is *not* a SystemVerilog string escape (only `\n \t \v \f \a \\` and octal/hex escapes are): VCS reads it as the letter `r`. Write a carriage return as `8'h0D`.

The trace is loaded into memory once and then replayed on every configuration; parsing a text file for every configuration would dominate the run time of the sweep.

2.3 Generators

`dv/model/traces/gen_trace.py` writes one trace per call. It takes the pattern, the number of accesses, the seed, `ADDR_W` and the pattern parameters, and writes the header line. Required patterns:

Pattern	Definition	What it exercises
`seq`	Reads with stride 1 over the whole address space ($2^{\texttt{ADDR_W}}$ words), repeated	Spatial locality; compulsory/capacity misses
`random`	Uniform random reads and writes over the whole address space, `wr_pct` writes	No locality; write-backs
`conflict`	Several tags on one index (stride = cache size in words), cycled	Conflict misses; associativity
`loop`	Repeated walk over an array slightly larger than the cache (for example 1.25 × capacity)	Replacement policy pathology
`fits`	Random reads and writes inside a working set smaller than the cache	Steady-state hits after warm-up
`mixed`	Interleaving of an array walk (60%), a stack-like hot region of 32 words with 50% writes (30%) and random accesses (10%)	A "realistic" program-like mix

These six traces are the "test names" of this lab: each is a directed workload whose expected behavior you can state before running it (`seq`: hit rate $1 - 1/W$; `conflict`: zero hits until there are as many ways as tags; `fits`: only cold misses), plus one (`mixed`) whose result you cannot predict by hand.

With `ADDR_W = 10` the address space has 1024 words: a configuration with a larger capacity (128 lines × 16 words) cannot be simulated by the model (the set and offset bits exceed `ADDR_W`) nor built in RTL (`TAG_W` < 1), so `tb_explore` skips it with a message.

```
# dv/model/traces/gen_trace.py (excerpt) -- generators are Python generators
def mixed(n, rng, addr_w, base_arr=0x000, arr_words=256, stack_top=0x3ff):
    i = 0
    for _ in range(n):
        r = rng.random()
        if r < 0.6:                                  # array walk
            yield ("R", base_arr + i % arr_words, None); i += 1
        elif r < 0.9:                                # stack-like hot region
            a = stack_top - rng.randrange(32)
            if rng.random() < 0.5:
                yield ("W", a, rng.getrandbits(32))
            else:
                yield ("R", a, None)
        else:                                        # random
            yield ("R", rng.getrandbits(addr_w), None)
```

Use `random.Random(seed)`, never the global `random` module state, so that two traces generated in one script do not influence each other. The standard trace set (all six patterns, 20 000 accesses, seed 1, `ADDR_W = 10`) is generated by `make traces` and committed, together with `dv/model/traces/MANIFEST`, a list of file names and SHA-256 checksums. Model and RTL must replay byte-identical files; the checksum goes into every CSV row.

3 Model-Only Exploration

3.1 The Exploration Testbench

Because the model has no clock and no RTL, it runs orders of magnitude faster than RTL simulation. `dv/model/explore/tb_explore.sv` is a model-only testbench (a module with one `initial` block, no clock):

```
module tb_explore;
  import cache_model_pkg::*;
  parameter int ADDR_W = 10, DW = 32;
  typedef cpu_txn #(ADDR_W, DW) txn_t;

  initial begin
    txn_t trace [$];  cache_cfg cfg, cfg_list [$];
    string tfile, cfgfile, csv;  int fd;
    if (!$value$plusargs("TRACE=%s", tfile))   $fatal(1, "+TRACE= missing");
    if (!$value$plusargs("CONFIGS=%s", cfgfile)) cfgfile = "configs.txt";
    if (!$value$plusargs("CSV=%s", csv))        csv = "explore.csv";
    trace_reader #(ADDR_W, DW)::load(tfile, trace);
    read_configs(cfgfile, cfg_list);            // one cache_cfg per line
    fd = $fopen(csv, "a");
    foreach (cfg_list[i]) begin
      automatic cache_model #(ADDR_W, DW) m;          // automatic: see below
      automatic mem_model   #(ADDR_W, DW) mem = new();
      automatic mem_txn     #(ADDR_W, DW) wbs [$];
      cfg = cfg_list[i];
      if (!cfg.is_valid()) continue;            // e.g. ways > num_lines,
                                                // or capacity > 2**ADDR_W words
      m = new(cfg, mem);
      foreach (trace[k]) void'(m.access(trace[k]));
```

```
      m.flush(wbs);
      // abbreviated: the full row also has sha256 and capacity_words
      $fdisplay(fd, "%s,%0d,%0d,%0d,%s,%0d,%0d,%0d,%0d", tfile,
                cfg.num_lines, cfg.line_words, cfg.ways, cfg.policy,
                m.stats.accesses, m.stats.hits, m.stats.misses,
                m.stats.writebacks);
    end
    $fclose(fd);
    $display("TEST PASSED");
  end
endmodule
```

Required sweep (`dv/model/explore/configs.txt`, one configuration per line, the same `key=value` style as `hw/ip/<ip>/syn/configs.txt` in P3):

- `num_lines` $\in \{8, 16, 32, 64, 128\}$, `line_words` $\in \{2, 4, 8, 16\}$, `ways` $\in \{1, 2, 4,$ fully associative$\}$, all three policies (with `ways = 1` only one policy row is needed);
- all six traces of Section 2.

That is 200 configuration lines, of which 190 fit the 1024-word address space of the traces. With VCS the whole sweep (190 configurations $\times$ 6 traces = 22.8 million model accesses) takes about 30 seconds.

Variables declared inside a loop of an `initial` block are *static* by default: without `automatic`, `mem = new()` would run once at time 0 and every configuration would share (and dirty) the same backing memory, and `wbs` would keep growing. The reference solution puts the whole body into a `function automatic` called from the `initial` block, which has the same effect for every declaration.

The CSV has a header line and one row per (trace, configuration): `trace, sha256, warmup, num_lines, line_words, ways, policy, capacity_words, accesses, hits, misses, writebacks, dirty_misses, flush_wb` (T-AM2 appends `stall_cycles, cycles, flush_cycles`). Derived quantities (hit rate, memory traffic) are computed in Python, not in the testbench: the CSV holds raw counts only, so that nothing is lost and any metric can be added later without re-running.

3.2 Cold Start and Warm-Up

A short trace on a large cache is dominated by *compulsory* misses, which say nothing about the steady state. Add `+WARMUP=n`: run the first n accesses, then clear the statistics (not the cache) and run the rest. Use `clone()` from T-MOD to warm a model once and run several experiments from the same warm state. State in every plot whether it is cold or warm.

3.3 The Three-C Classification

The model can classify every miss, which the RTL cannot:

- **Compulsory**: the first access to a line ever. Equal to the number of distinct lines in the trace; independent of the cache.
- **Capacity**: misses of a *fully associative* LRU cache of the same capacity, minus the compulsory misses.
- **Conflict**: misses of the configuration under study minus the misses of the fully associative cache of the same capacity.

$$M_{\text{cfg}} = \underbrace{M_{\text{comp}}}_{\text{distinct lines}} + \underbrace{(M_{\text{FA}} - M_{\text{comp}})}_{\text{capacity}} + \underbrace{(M_{\text{cfg}} - M_{\text{FA}})}_{\text{conflict}}$$

The fully associative configuration is the model with `ways = num_lines` (one set); no new code is needed. The conflict term can be negative for some patterns (a direct-mapped cache can beat fully associative LRU on a loop): explain any negative value you find.

3.4 Plots

`dv/model/explore/plot_explore.py` reads the CSV (Python standard `csv` module and `matplotlib`) and writes PNG files to `dv/model/explore/results/`. Run it with `$(PYTHON)` from `env.sh`. If `matplotlib` is not installed (as on the Synopsys cloud's `python3.11`), do not install packages system-wide: either use a private virtual environment, or write the few chart types you need as SVG text from Python and convert them with `rsvg-convert` (the reference solution does both behind one small plotting module). Required plots:

1. Hit rate versus capacity (words), one line per `line_words`, direct-mapped, one panel per trace.
2. Hit rate versus `ways` and policy at fixed capacity (64 words), one group of bars per trace.
3. Three-C stacked bars for the default R4 configuration on every trace.
4. Memory traffic in words per access, $(misses + writebacks) \cdot line_words / accesses$, versus `line_words`.
5. The Pareto plot of the memo: hit rate (`mixed` trace) versus area (Section 6).

Every plot has labeled axes with units, a title that names the trace and the warm/cold state, and is regenerated by one `make plots`; no plot is edited by hand.

3.5 Questions

Answer in `dv/model/explore/README.md`, with the plot or table that supports each answer:

1. For a fixed capacity of 64 words, which configuration has the best hit rate on each trace? Is there a single best one?
2. With the same total capacity, compare `line_words=2, num_lines=32` against `line_words=8, num_lines=8` on the `seq` and `random` traces. Why do longer lines help one pattern and hurt the other? What happens to memory traffic?
3. On the `loop` pattern, LRU performs badly. Explain why, and which policy does better. Would the answer change if the array were smaller than the cache?
4. Classify the misses of the default R4 configuration with the three-C model on every trace. Which kind dominates on `mixed`, and which parameter would you change to reduce it?
5. On `conflict`, how many ways remove all conflict misses? Relate the answer to the number of tags the generator cycles through.
6. Compare the model's hit rate on `seq`, `fits`, `conflict` and `random` with your hand predictions from Lab R4 and the measured values from Lab V3. Explain every difference larger than one percentage point.

4 Model–RTL Correlation

4.1 What Must Match, and Why Exactly

For the configurations the RTL supports (direct-mapped; 2-way LRU if you built the R4 stretch goal), the model and the RTL implement the same architecture. Hits, misses and write-backs are *functional* quantities: they do not depend on timing, so they must agree exactly. Any difference is a bug — in the model, in the RTL, in the trace replay or in the counter definitions — and must be explained before any exploration result is used. (Cycle counts are a different matter: they are correlated with a tolerance in T-AM2.)

4.2 The Trace-Driven RTL Testbench

`dv/model/explore/tb_replay.sv` is a testbench top for the R4 RTL that reads the same trace file. It is deliberately simple: it is a measurement harness, not a verification environment (that is T-DV1).

Figure 2 shows its structure.

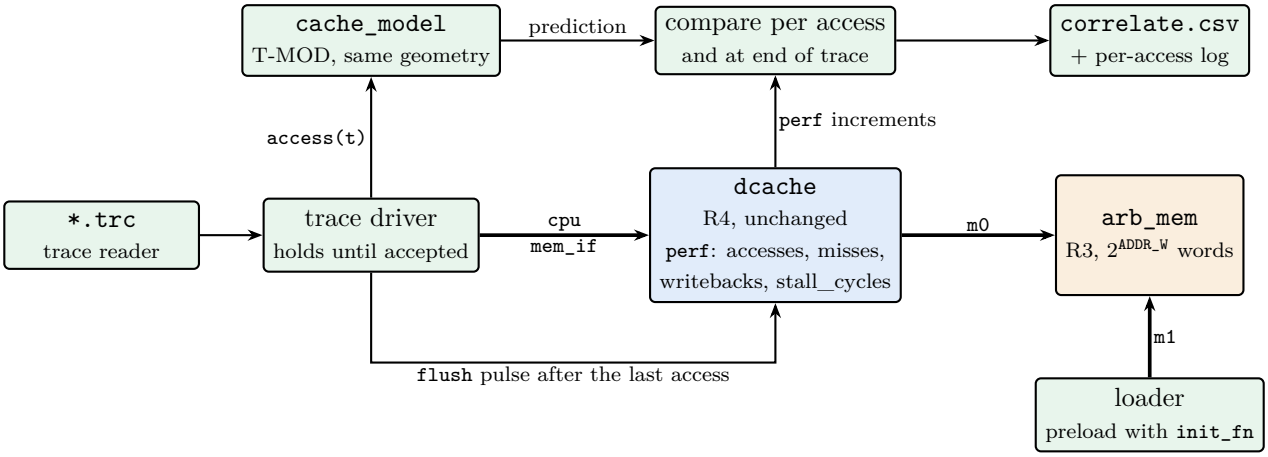


Figure 2: The trace-driven RTL testbench `tb_replay`: the R4 cache (blue) with `arb_mem` as main memory (orange), driven from the trace while the T-MOD model runs in lockstep (testbench components green). The model's `cache_cfg` is built from the testbench parameters.

1. The loader preloads the memory with the V3 pattern through port `m1`; the model's `mem_model` uses the same `init_fn`. The loader is idle afterwards.
2. For every trace access, the driver calls `model.access(t)` and keeps the prediction, then drives the request on the CPU-side `mem_if` and holds it until it is accepted (protocol of Lab R3). Requests are issued back to back.
3. At every acceptance, the driver samples the `perf` counters. The increments since the previous acceptance classify what the RTL did: no increment of `misses` is a hit; `misses` +1 is a clean miss; `misses` and `writebacks` +1 is a dirty miss. The driver compares this with the model's result for the same access, and stops at the *first divergent access* with the access number, the trace line, the model's `convert2string()` of the set, and both counter values. Note that the counters are sampled *at* the acceptance edge, before that edge updates them: `accesses` then equals the number of *earlier* acceptances (a check that no trace line was skipped or duplicated).
4. After the last access, the testbench pulses `flush`, waits for `flush_done`, calls `model.flush()` and compares the four end-of-trace values: `accesses`, `misses`, `writebacks` (including the flush) and the hit count *accesses* − *misses*.
5. It appends one row to `correlate.csv` and prints `TEST PASSED` only if everything matched. It

also writes a per-access log (access number, result, acceptance cycle, stall cycles of that access; the reference solution also logs the cycles of the first memory request, the last refill read and the last read response of a miss) for use in Lab T-AM2.

```
// classify the RTL behavior of one access from the counter increments
function automatic result_e rtl_result(dcache_pkg::perf_t now, dcache_pkg::perf_t prev);
  if (now.misses == prev.misses)                return HIT;
  else if (now.writebacks == prev.writebacks)   return MISS_CLEAN;
  else                                          return MISS_DIRTY;
endfunction
```

The design parameters of the RTL are compile-time: `tb_replay` passes `NUM_LINES` and `LINE_WORDS` with `PARAMS`, and the model's `cache_cfg` is set from the same values (`tb_replay` passes its own parameters into `cfg`, so the two cannot disagree). Mind the names: the RTL's `NUM_LINES` counts *sets*, the model's `num_lines` counts *lines*, so `cfg.num_lines = NUM_LINES * WAYS`. Run at least: the default configuration and two others from the P3 sweep, on all six traces (the reference solution adds the 2-way stretch configuration: 24 runs, all exact).

Trace	Config	Hits model	Hits RTL	Misses model	Misses RTL	WB model	WB RTL	Match
`seq`	16×4							
`random`	16×4							
`mixed`	16×4							
...								

4.3 When They Disagree

Typical causes, in the order you should suspect them:

- **Counter definitions**: the RTL counts a miss when the refill starts, the model when the access is processed; the RTL counts write-backs during flush separately; `accesses` counts a held request more than once. Fix the definitions to those of R4, then re-check both sides.
- **Stimulus**: a trace line skipped or duplicated by the driver (compare `accesses` first), or the RTL replaying a different file (checksum).
- **Initial state**: the memory preload differs from the model's `init_fn` — shows up as data, not as counts, but check it.
- **A real bug**: in the model (re-run the T-MOD hand traces with the failing pattern) or in the RTL (write a directed test, fix the RTL with a commit referencing the trace, and re-run the V3 regression).

Record every disagreement you found and its root cause in the README. A correlation report with "all match" and no history of mismatches is less convincing than one that shows what was found and fixed.

Test term — Directed test. A hand-written stimulus with a known expected result, aimed at one behavior. When the correlation fails, cut the trace down to the few accesses around the first divergent access and replay them on the model's hand-trace testbench (T-MOD) and on the R4 smoke testbench: a ten-line directed test is far easier to debug than 20 000 accesses, and it becomes a permanent check.

Test term — Negative / error-injection test. Break something on purpose and show that the check fails. Before trusting an "all match", copy `dcache.sv` to a scratch directory, insert a bug (for example: a write hit does not set the dirty bit), point `TB_FLIST` at the copy and replay `mixed`: `tb_replay` must stop at the first divergent access (in the reference solution access 50: model `MISS_DIRTY`, RTL `MISS_CLEAN`). A correlation harness that has never failed proves nothing.

4.4 Model Speed versus RTL Speed

Measure the wall-clock time of the `mixed` trace (scale it to 10^6 accesses) on `tb_explore` with one configuration, and on `tb_replay`. Report accesses per second for each, excluding and including the build time (VCS or Verilator), and the ratio. A single configuration on a 10^6-line text trace mostly measures the trace parser; report the per-configuration model throughput of the sweep as well. Then answer: how long would the full sweep of Section 3 take on RTL, including one build per configuration? What does that ratio mean for architecture work, and for the choice between "change a parameter" and "change a model"?

5 Performance and AMAT

The model gives hit rates; the RTL replay also gives the number of cycles. Recall from Lab R4:

$$AMAT = hit\ time + miss\ rate \times miss\ penalty$$

With a blocking write-back cache the miss penalty is not one number: a clean miss costs one refill, a dirty miss costs a write-back plus a refill. With m_c and m_d the fractions of accesses that are clean and dirty misses, and P_c, P_d their penalties in cycles:

$$AMAT = t_{\text{hit}} + m_c\,P_c + m_d\,P_d, \qquad m_c + m_d = miss\ rate.$$

1. Measure P_c and P_d on the RTL for `line_words` $\in \{2, 4, 8\}$ from the per-access log of `tb_replay` (stall cycles of single clean and dirty misses). How do they depend on `line_words`? Fit a straight line.
2. Compute the AMAT of every trace from the model's m_c, m_d and the measured penalties, and compare with the RTL's average latency $1 + stall_cycles/accesses$ from the R4 counters. Does the measured average latency match the formula? If not, which cycles does the formula not count?
3. Fill in the table below for the default configuration (20 000 accesses; the RTL replay starts from reset, so compare the cold model numbers with it, and quote the warm ones separately).

Trace	Hit rate (R4 hand)	Hit rate (model)	Hit rate (RTL)	Write-backs	AMAT formula	Avg. latency RTL
`seq`						
`fits`						
`conflict`						
`random`						
`mixed`	—					

This is the bridge to Lab T-AM2, which turns these penalties into calibrated parameters of a timing model and predicts cycles for any configuration without running the RTL.

6 The Recommendation Memo

6.1 Area Data

Lab P3 synthesized `dcache` for several `NUM_LINES` and `LINE_WORDS` values with `make sweep`. Take that CSV (from `build/dcache/` or the table in `hw/ip/dcache/doc/ppa.md`) and fit an area model, since the sweep did not synthesize every configuration you explored:

$$A(N, W) \approx A_0 + N \cdot (W \cdot DW \cdot a_d + (TAG_W + 2) \cdot a_t),$$

with $N =$ `num_lines`, $W =$ `line_words`, a_d, a_t the area per data and per tag/control bit (flip-flop plus its share of the multiplexers) and A_0 the controller. Fit A_0, a_d, a_t by least squares, check the fit on one P3 configuration you did *not* use for fitting, and state the error. Note that `TAG_W` shrinks as the cache grows. If you built the 2-way stretch, add a term for the second comparator and the LRU bits.

6.2 The Memo

The coordinator gives an area budget (for example: at most 1.5× the area of the default configuration in P3) and a workload weighting (for example: 70% `mixed`, 30% `seq`). Write `dv/model/explore/doc/recommendation.md`, one page, for an architecture review:

```
Question      The cache configuration for <workload> within <budget> um^2.
Recommendation   NUM_LINES=.., LINE_WORDS=.., WAYS=..   (one line)
Evidence      Pareto plot (hit rate vs area), the three best candidates in a table
              with hit rate per trace, weighted AMAT, traffic and area.
Why not       The runner-up and the "obvious" choice (largest cache that fits),
              and why they lose.
Confidence    Correlation status (exact on N traces x M configs), area-model error,
              trace limitations (synthetic, 20k accesses, no instruction fetch).
Risks         Timing: critical path through tag compare grows with ways (P3);
              sensitivity of the choice to the workload weighting.
Next step     What T-AM2 (cycles) or a real-software trace would need to confirm.
```

A configuration is a candidate only if it is *Pareto-optimal*: no other configuration has both less area and a better weighted metric. Show the Pareto front on the plot and choose on it. “The biggest cache that fits the budget” is a valid answer only if you show that nothing smaller is within the noise of it.

7 Requirements and Tasks

1. Write `dv/model/traces/gen_trace.py` with the six patterns, the header line and seeded generators; generate the standard trace set and the `MANIFEST` with `make traces`.
2. Write the trace reader class and `tb_explore`; run the required sweep on all traces (cold and warm) into `explore.csv`.
3. Write `plot_explore.py` and produce the five required plots with `make plots`.
4. Answer the six exploration questions in the README.
5. Write `tb_replay`; replay all six traces on the R4 RTL for the default and two other configurations; obtain an exact match of hits, misses and write-backs; document every mismatch found on the way and its root cause.
6. Measure model and RTL throughput and answer the speed questions.

7. Measure clean and dirty miss penalties on the RTL, compute AMAT, and fill in the AMAT table.
8. Fit the area model to the P3 data and check it on a held-out configuration.
9. Write the one-page recommendation memo and present it in a ten-minute review with the coordinator and the architecture-modeling team.
10. The model package and the RTL are not modified in this lab. If a bug is found in either, fix it on its own branch with a commit referencing the failing trace, and re-run the T-MOD unit tests or the V3 regression.

Test term — Regression. All tests × seeds × parameter sets, run by one command with a pass/fail summary. Here two regressions guard the two things this lab relies on: the T-MOD unit tests (`make -C dv/model/cache_model regress`) for the model and the V3 regression (`make -C hw/ip/dcache regress`) for the RTL. A fix on either side is only done when its regression passes again — and `make correlate` is itself a small regression (6 traces × the replay configurations).

8 Repository and Flow

```
dv/model/traces/
+-- gen_trace.py              trace generators (seeded, header with all parameters)
+-- MANIFEST                  file name + SHA-256 of every committed trace
+-- seq.trc random.trc conflict.trc loop.trc fits.trc mixed.trc
dv/model/explore/
+-- README.md                 questions, correlation history, speed, AMAT table
+-- Makefile                  IP := explore; traces, explore, plots, replay, correlate
+-- trace_reader.svh          shared trace reader (included by both tops)
+-- tb_explore.sv             model-only sweep top
+-- tb_explore.f
+-- tb_replay.sv              trace-driven RTL top: dcache + arb_mem + model
+-- tb_replay.f
+-- configs.txt               model configurations for the sweep
+-- replay_configs.txt        RTL PARAMS sets for correlation
+-- plot_explore.py  correlate.py  area_fit.py
+-- lint/                     lint wrapper instantiating both tops (make lint)
+-- results/                  final CSV files and PNG plots (small, committed)
+-- doc/recommendation.md     the one-page memo
```

```
// dv/model/explore/tb_replay.f
-f ${REPO_ROOT}/dv/model/cache_model/cache_model.f
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
-f ${REPO_ROOT}/hw/ip/dcache/rtl/dcache.f
${REPO_ROOT}/dv/common/tb_utils_pkg.sv
+incdir+${REPO_ROOT}/dv/model/explore
${REPO_ROOT}/dv/model/explore/tb_replay.sv
```

`dv/model/` is not under `hw/ip/`, but the Makefile follows the same contract (`IP := explore`, variables first, then the shared `common.mk` and `lint.mk`). Two tops live in one directory, so it sets `SIM_PER_TB := 1`: each `TB_TOP` gets its own model and logs under `build/explore/<cfg>/sim/<TB_TOP>/`, and `TB_FLIST` follows `TB_TOP`. `tb_explore` is always run with an empty `PARAMS` (its configurations are run-time objects); `PARAMS` is the `dcache` geometry of `tb_replay`.

The Makefile reuses the shared flow fragments (`build`, `sim`, `clean`, `PARAMS`, `PLUSARGS`) and adds five convenience targets local to `dv/model/explore/`:

Target	Function
`traces`	Run `gen_trace.py` for the standard set, rewrite `MANIFEST`
`explore`	`sim` with `TB_TOP=tb_explore` for every trace in `TRACES` (`WARMUP=n`); collect `explore_w<n>.csv`
`plots`	Run `plot_explore.py` on the CSV files, write `results/*.png`
`replay`	`sim` with `TB_TOP=tb_replay` for one `TRACE` and one `PARAMS` set
`correlate`	`replay` over all traces × `replay_configs.txt`; `correlate.py` prints the match table

```
source env.sh
make -C dv/model/explore traces
make -C dv/model/explore explore TRACES="seq mixed" WARMUP=2000
make -C dv/model/explore plots
make -C dv/model/explore replay TRACE=mixed PARAMS="NUM_LINES=32 LINE_WORDS=4"
make -C dv/model/explore correlate          # exit code != 0 on any mismatch
```

On the Synopsys cloud: Both tops are simulated with VCS (`SIM=vcs`, the cloud default): the class-based model needs a full SystemVerilog class implementation, and Verilator 5.013 cannot compile the T-MOD package (see Lab T-MOD). VCS compiles the model to native code, so the model-only top runs much faster than the RTL replay — measured in the reference solution on 10^6 `mixed` accesses: about $1.5 \cdot 10^6$ model accesses per second per configuration in the sweep versus about $9 \cdot 10^4$ accesses per second for `tb_replay` (RTL + model), and 30 s for the full model sweep versus an estimated 20 minutes on RTL (one 5.5 s VCS build per configuration). Lint the tops with `make lint` (the Makefile selects `LINT_TOOL=vcs`, VCS `+lint=all`: SpyGlass and VC SpyGlass read synthesizable RTL only). Waveforms of a replay: `make sim TB_TOP=tb_replay TRACE=mixed WAVES=1`, then `make waves` (Verdi, FSDB).

Test term — Sanity test (`make check-all`). The smoke test of every block run together, to check that a change did not break anything that used to work. The top-level `check-all` runs `lint` and `sim` of every `dv/model/*/Makefile`; the default `sim` of `dv/model/explore` is the replay of `seq` on the default configuration, so from now on an RTL change that breaks the model–RTL correlation breaks `check-all`.

Outputs go to `build/explore/<cfg>/sim/<TB_TOP>/` (the collected CSV files to `build/explore/`); only the final CSV files and plots are copied to `dv/model/explore/results/`. Make `replay TRACE=seq` the default `sim` of the Makefile (`TB_TOP ?= tb_replay`, `TRACE ?= seq`), so that `check-all` runs it. Git: branch `lab/tam1-dse`, tag `tam1-done`.

9 Deliverables

The merge request contains the generators, the committed traces and `MANIFEST`, the two testbench tops, the scripts, the results and:

- `dv/model/explore/README.md`: how to run every target; the trace descriptions; the answers to the exploration questions with plots; the correlation table and the history of mismatches and their root causes; the speed comparison; the AMAT table and the miss-penalty fit.
- `dv/model/explore/doc/recommendation.md`: the one-page memo.
- `dv/model/traces/` header lines that make every trace reproducible.
- The slides (or the memo itself) used in the review, in `doc/reviews/`.

10 Assessment

Criterion	Weight
Trace format, generators, reproducibility	10%
Model-only sweep, CSV and plots	15%
Exploration questions, three-C analysis and explanations	20%
Model–RTL correlation: exact match, mismatch history	25%
Speed comparison and AMAT analysis	10%
Area model and recommendation memo	20%
Total	100%

11 Review Questions

1. Why must every trace header contain the generator version, the seed and all parameters? What goes wrong in a review if it does not?
2. Why does `tb_explore` write raw counts and not hit rates?
3. Why do hits, misses and write-backs have to match the RTL exactly, while cycles in T-AM2 only have to match within a tolerance?
4. Your correlation shows one extra miss in the RTL on 20 000 accesses. List the possible causes in the order you would check them.
5. How does the replay testbench find the *first* divergent access, and why is that much more useful than an end-of-trace comparison?
6. Why does LRU perform badly on a loop slightly larger than the cache? Which policy is better, and why?
7. How do you measure conflict misses with a model, and why can the RTL not tell you?
8. With the same capacity, why do longer lines help `seq` and hurt `random`? What happens to memory traffic?
9. What does the speed ratio between model and RTL mean for the number of configurations an architect can evaluate before an RTL freeze?
10. What does your memo say about the configuration you chose in Lab R4?

12 References

- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*: Appendix B and Chapter 2 (cache performance, the three-C model, AMAT).
- M. D. Hill and A. J. Smith, “Evaluating Associativity in CPU Caches”, *IEEE Transactions on Computers*, 1989 (the three-C classification).
- R. A. Uhlig and T. N. Mudge, “Trace-Driven Memory Simulation: A Survey”, *ACM Computing Surveys*, 1997.
- Dinero IV trace-driven cache simulator (University of Wisconsin): a classic reference for trace formats and exploration.

- CVA6 repository (OpenHW Group), `perf-model/`: an industrial example of a model correlated against RTL traces (used in T-AM2).
- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: memory systems chapter.
- Python documentation: `random.Random`, `csv`, `hashlib`; matplotlib user guide.